

# StreetSignal Platform

## Base de Datos MySQL - Documentación del Esquema

| Atributo            | Valor                        |
|---------------------|------------------------------|
| Versión             | 1.0.0                        |
| Motor               | MySQL 8.0+ / MariaDB 10.6+   |
| Juego de caracteres | utf8mb4 / utf8mb4_unicode_ci |
| Tablas              | 9                            |
| Vistas              | 0 (lógica en el backend)     |
| Procedimientos      | 0 (lógica en el backend)     |
| Disparadores        | 0 (lógica en el backend)     |
| Llaves foráneas     | 11                           |
| Índices             | 23 (custom) + PK/UK          |

**Alcance:** Este documento describe el esquema relacional que respalda la plataforma StreetSignal de reportes ciudadanos. Se deriva del contrato OpenAPI (streetsignal-api-contracts.yml) y del plan de proyecto (Plan\_TODO\_Plataforma\_Reportes\_Ciudadanos.docx).

**Audiencia:** Desarrolladores backend, DBAs, revisores, evaluadores.

# 1. Diagrama Entidad-Relación

El diagrama a continuación muestra las 9 tablas y sus relaciones. La cardinalidad sigue la notación de pata de gallo: | = exactamente uno, o = cero o muchos, } = uno o muchos.

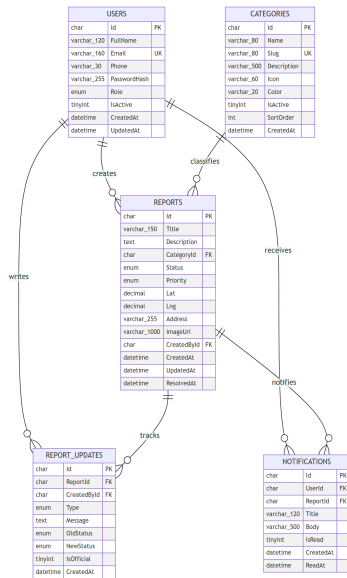


Figura 1.1 - Diagrama ER de la base de datos StreetSignal

## 2. Referencia de Tablas

Cada tabla se lista a continuación con sus columnas, tipos de datos, restricciones y una descripción breve de su propósito. PK = clave primaria, FK = clave foránea, UK = clave única. NN = NOT NULL.

### 2.1 Users

Usuarios de la plataforma: ciudadanos, funcionarios y administradores. Soporta autenticación JWT y los 3 roles definidos en el contrato (Citizen, Staff, Admin).

| Columna      | Tipo         | Nulo | Clave | Notas                              |
|--------------|--------------|------|-------|------------------------------------|
| Id           | CHAR(36)     | NN   | PK    | UUID v4                            |
| FullName     | VARCHAR(120) | NN   |       |                                    |
| Email        | VARCHAR(160) | NN   | UK    | Unico, lowercased antes de guardar |
| Phone        | VARCHAR(30)  | Y    |       | Opcional                           |
| PasswordHash | VARCHAR(255) | NN   |       | BCrypt/Argon2, nunca en plano      |
| Role         | ENUM         | NN   | IDX   | Citizen   Staff   Admin            |
| IsActive     | TINYINT(1)   | NN   | IDX   | Soft-delete                        |
| LastLoginAt  | DATETIME(3)  | Y    |       | Actualizado en /auth/login         |
| CreatedAt    | DATETIME(3)  | NN   | IDX   | DEFAULT CURRENT_TIMESTAMP(3)       |
| UpdatedAt    | DATETIME(3)  | Y    |       | ON UPDATE CURRENT_TIMESTAMP(3)     |

### 2.2 Categories

Catalogo de categorias seleccionables al crear un reporte. Soporta soft-delete (IsActive) para ocultar sin perder historial.

| Columna     | Tipo         | Nulo | Clave | Notas                      |
|-------------|--------------|------|-------|----------------------------|
| Id          | CHAR(36)     | NN   | PK    | UUID v4                    |
| Name        | VARCHAR(80)  | NN   |       | Nombre legible             |
| Slug        | VARCHAR(80)  | NN   | UK    | Identificador URL-friendly |
| Description | VARCHAR(500) | Y    |       |                            |
| Icon        | VARCHAR(60)  | Y    |       | Nombre del icono UI        |
| Color       | VARCHAR(20)  | Y    |       | Hex color #RRGGBB          |
| IsActive    | TINYINT(1)   | NN   | IDX   | Soft-hide                  |
| SortOrder   | INT          | NN   | IDX   | Orden de despliegue        |
| CreatedAt   | DATETIME(3)  | NN   |       |                            |
| UpdatedAt   | DATETIME(3)  | Y    |       |                            |

## 2.3 Reports

Reportes ciudadanos. Tabla central del sistema. Soporta geolocalizacion, prioridad interna, asignacion a staff, e imagen principal. Multiples imagenes viven en ReportImages.

| Column          | Type          | Null | Key | Notes                             |
|-----------------|---------------|------|-----|-----------------------------------|
| Id              | CHAR(36)      | NN   | PK  |                                   |
| Title           | VARCHAR(150)  | NN   | FT  | Full-text search                  |
| Description     | TEXT          | NN   | FT  |                                   |
| CategoryId      | CHAR(36)      | NN   | FK  | -> Categories(Id)                 |
| Status          | ENUM          | NN   | IDX | 6 valores posibles                |
| Priority        | ENUM          | Y    |     | Low/Medium/High/Critical          |
| Latitude        | DECIMAL(10,7) | NN   | GEO | CHECK -90..90                     |
| Longitude       | DECIMAL(10,7) | NN   | GEO | CHECK -180..180                   |
| Address         | VARCHAR(255)  | Y    | FT  | Direccion humana                  |
| ImageUrl        | VARCHAR(1000) | Y    |     | Imagen principal (compat API)     |
| RejectionReason | VARCHAR(500)  | Y    |     | Solo si Status=Rejected           |
| CreatedById     | CHAR(36)      | NN   | FK  | -> Users(Id)                      |
| AssignedToId    | CHAR(36)      | Y    | FK  | -> Users(Id), NULL si sin asignar |
| CreatedAt       | DATETIME(3)   | NN   | IDX |                                   |
| UpdatedAt       | DATETIME(3)   | Y    |     |                                   |
| ResolvedAt      | DATETIME(3)   | Y    |     | Auto-set al pasar a Resolved      |

## 2.4 ReportImages

Galeria de imagenes adjuntas a un reporte. Permite multiples archivos por incidente con orden configurable y validacion de tamano (max 20 MB).

| Columna     | Tipo          | Nulo | Clave | Notas                             |
|-------------|---------------|------|-------|-----------------------------------|
| Id          | CHAR(36)      | NN   | PK    |                                   |
| ReportId    | CHAR(36)      | NN   | FK    | -> Reports(Id), ON DELETE CASCADE |
| FileName    | VARCHAR(180)  | NN   |       |                                   |
| ContentType | VARCHAR(80)   | NN   |       | image/jpeg, image/png, image/webp |
| FileSize    | BIGINT        | NN   |       | CHECK 0 < size <= 20MB            |
| Url         | VARCHAR(1000) | NN   |       | Ruta publica o URL CDN            |

| Columna   | Tipo        | Nulo | Clave | Notas                  |
|-----------|-------------|------|-------|------------------------|
| SortOrder | INT         | NN   | IDX   | Orden de visualizacion |
| CreatedAt | DATETIME(3) | NN   |       |                        |

## 2.5 ReportUpdates

Timeline / bitacora inmutable por reporte. Modela los 3 tipos de eventos del contrato: Comment, StatusChange, System. CHECK garantiza que un StatusChange tenga OldStatus y NewStatus distintos.

| Column      | Type        | Null | Key | Notes                             |
|-------------|-------------|------|-----|-----------------------------------|
| Id          | CHAR(36)    | NN   | PK  |                                   |
| ReportId    | CHAR(36)    | NN   | FK  | -> Reports(Id), ON DELETE CASCADE |
| CreatedById | CHAR(36)    | NN   | FK  | -> Users(Id)                      |
| Type        | ENUM        | NN   | IDX | Comment   StatusChange   System   |
| Message     | TEXT        | Y    |     | Comentario o descripcion          |
| OldStatus   | VARCHAR(30) | Y    |     | Solo si Type=StatusChange         |
| NewStatus   | VARCHAR(30) | Y    |     | Solo si Type=StatusChange         |
| IsOfficial  | TINYINT(1)  | NN   |     | 1 si lo escribe Staff/Admin       |
| CreatedAt   | DATETIME(3) | NN   | IDX |                                   |

## 2.6 DeviceTokens

Tokens push de dispositivos moviles. Multiples tokens por usuario (un usuario puede tener varios telefonos). Unico (UserId, Token) para evitar duplicados.

| Column    | Type         | Null | Key | Notes                           |
|-----------|--------------|------|-----|---------------------------------|
| Id        | CHAR(36)     | NN   | PK  |                                 |
| UserId    | CHAR(36)     | NN   | FK  | -> Users(Id), ON DELETE CASCADE |
| Token     | VARCHAR(500) | NN   | UK* | Expo/FCM/APNs                   |
| Platform  | ENUM         | NN   |     | Android   iOS   Web   Unknown   |
| IsActive  | TINYINT(1)   | NN   | IDX | Para invalidar al logout        |
| CreatedAt | DATETIME(3)  | NN   |     |                                 |
| UpdatedAt | DATETIME(3)  | Y    |     |                                 |

## 2.7 Notifications

Bandeja de notificaciones in-app. Generadas por el backend cuando se cambia el estado de un reporte o se agrega un comentario oficial. Indexadas para consultas rápidas por usuario.

| Column    | Type         | Null | Key | Notes                             |
|-----------|--------------|------|-----|-----------------------------------|
| Id        | CHAR(36)     | NN   | PK  |                                   |
| UserId    | CHAR(36)     | NN   | FK  | -> Users(Id), ON DELETE CASCADE   |
| ReportId  | CHAR(36)     | Y    | FK  | -> Reports(Id), ON DELETE CASCADE |
| Title     | VARCHAR(120) | NN   |     |                                   |
| Body      | VARCHAR(500) | NN   |     |                                   |
| Type      | ENUM         | NN   |     | 6 tipos (StatusChanged, ...)      |
| IsRead    | TINYINT(1)   | NN   | IDX | 0=unread, 1=read                  |
| ReadAt    | DATETIME(3)  | Y    |     |                                   |
| CreatedAt | DATETIME(3)  | NN   | IDX |                                   |

## 2.8 AuditLogs

Bitacora inmutable para defensa tecnica. Registra acciones criticas (login, status\_changed, report\_created, etc.) con metadata estructurada en JSON.

| Columna     | Tipo         | Nulo | Clave | Notas                             |
|-------------|--------------|------|-------|-----------------------------------|
| Id          | CHAR(36)     | NN   | PK    |                                   |
| UserId      | CHAR(36)     | Y    | FK    | NULL si fue sistema               |
| Action      | VARCHAR(80)  | NN   | IDX   | report_created, status_changed... |
| EntityName  | VARCHAR(80)  | NN   | IDX   | Report, User, Notification...     |
| EntityId    | VARCHAR(80)  | Y    |       |                                   |
| Description | VARCHAR(500) | Y    |       | Resumen humano                    |
| IpAddress   | VARCHAR(45)  | Y    |       | IPv4 o IPv6                       |
| UserAgent   | VARCHAR(500) | Y    |       |                                   |
| Metadata    | JSON         | Y    |       | Payload estructurado              |
| CreatedAt   | DATETIME(3)  | NN   | IDX   |                                   |

## 2.9 RefreshTokens

Tokens de refresco para JWT (opcional pero recomendado). Se guarda el SHA-256 del token, no el token en plano. Permite revocacion inmediata sin esperar a la expiracion.

| Column | Type     | Null | Key | Notes |
|--------|----------|------|-----|-------|
| Id     | CHAR(36) | NN   | PK  |       |

| Column      | Type         | Null | Key | Notes                           |
|-------------|--------------|------|-----|---------------------------------|
| UserId      | CHAR(36)     | NN   | FK  | -> Users(Id), ON DELETE CASCADE |
| TokenHash   | CHAR(64)     | NN   | UK  | SHA-256 del token               |
| ExpiresAt   | DATETIME     | NN   | IDX |                                 |
| RevokedAt   | DATETIME     | Y    |     | NULL = activo                   |
| CreatedByIp | VARCHAR(45)  | Y    |     |                                 |
| UserAgent   | VARCHAR(500) | Y    |     |                                 |
| CreatedAt   | DATETIME(3)  | NN   |     |                                 |

## 3. Vistas, Procedimientos y Disparadores

Por decision de arquitectura, la base de datos **no incluye vistas, stored procedures ni triggers**. Toda esa capa de logica vive en el backend .NET 8 (C# / Entity Framework Core / Dapper). Esta es una eleccion consciente, no una omision.

### 3.1 ¿Por qué?

- Versionado: la logica de negocio viaja junto al codigo C# en el mismo repositorio, con los mismos PRs, code reviews y branches.
- Testabilidad: las reglas (cambio de estado, validaciones, notificaciones) se prueban con xUnit + EF Core InMemory o WebApplicationFactory, no se ejecutan contra MySQL.
- Portabilidad: si mas adelante se migra a SQL Server o PostgreSQL, no hay que reescribir SPs ni adaptar triggers.
- Debugging: stack traces reales, breakpoints, logs estructurados, IntelliSense.
- Transacciones: EF Core maneja SaveChanges() / BeginTransaction() de forma explicita y mucho mas legible que un SP con SIGNAL SQLSTATE.

### 3.2 Qué vive en el backend (referencia)

| Capa           | Dónde                               | Qué hace                                                                      |
|----------------|-------------------------------------|-------------------------------------------------------------------------------|
| Service        | Services/ReportService.cs           | ChangeStatusAsync: valida transicion, abre transaccion, actualiza Reports, in |
| Service        | Services/NotificationService.cs     | CreateForUserAsync, MarkAsReadAsync, GetUnreadCountAsync.                     |
| Service        | Services/AuthService.cs             | RegisterAsync, LoginAsync, RefreshTokenAsync, hash con BCrypt, genera J       |
| Service        | Services/DeviceTokenService.cs      | UpsertAsync: inserta o reactiva token push por usuario/dispositivo.           |
| Interceptor    | Interceptors/AuditLogInterceptor.cs | SaveChanges interceptor que registra login_success, report_created, status_   |
| Service / LINQ | Services/Reports/ReportQueries.cs   | ListMyAsync, ListForStaffAsync, KPIs (ReportsByStatus, ReportsByCategory      |

### 3.3 Excepción: cuándo sí justifica agregar algo en la DB

Si en algun momento una consulta se vuelve cuellos de botella y se necesita agregaciones materializadas o reglas en el motor, se puede agregar en un archivo separado (*streetsignal\_db\_views.sql* o *streetsignal\_db\_jobs.sql*), siempre documentando la razon. Ejemplos validos:

- Una vista materializada refrescada cada 5 minutos para un dashboard con millones de filas.
- Un EVENT scheduler que limpia notificaciones de mas de 90 dias (politica de retencion).
- Un trigger BEFORE INSERT que normalice un campo calculado y sea requerido por rendimiento.



## 4. Enumeraciones y Máquinas de Estado

### 4.1 ReportStatus

Diagrama de transiciones validas para el campo **Reports.Status**. El servicio backend no valida transiciones específicas (cualquier cambio es permitido), pero la convención operativa es:

| Desde      | Hacia      | Quién | Mensaje esperado                              |
|------------|------------|-------|-----------------------------------------------|
| Pending    | InReview   | Staff | Asignado para revision.                       |
| InReview   | Assigned   | Staff | Asignado a cuadrilla.                         |
| Assigned   | InProgress | Staff | Cuadrilla en camino o trabajando.             |
| InProgress | Resolved   | Staff | Trabajo finalizado. ResolvedAt se setea auto. |
| InProgress | Rejected   | Staff | No procede. RejectionReason obligatorio.      |
| Pending    | Rejected   | Staff | Rechazo temprano por duplicado o invalido.    |

### 4.2 Otras Enumeraciones

| Enumeración | Campo                 | Valores permitidos                                                  |
|-------------|-----------------------|---------------------------------------------------------------------|
| UserRole    | Users.Role            | Citizen, Staff, Admin                                               |
| Priority    | Reports.Priority      | Low, Medium, High, Critical (nullable)                              |
| Platform    | DeviceTokens.Platform | Android, iOS, Web, Unknown                                          |
| UpdateType  | ReportUpdates.Type    | Comment, StatusChange, System                                       |
| NotifType   | Notifications.Type    | StatusChanged, NewComment, ReportAssigned, ReportResolved, ReportRe |

## 5. Consultas de Ejemplo

Consultas utiles para el backend .NET, el panel staff, y los dashboards.

### Listar mis reportes con categoria aplanada

```
SELECT r.Id, r.Title, r.Status, r.Priority,
       r.Latitude, r.Longitude, r.Address, r.ImageUrl,
       r.CreatedAt, r.UpdatedAt, r.ResolvedAt,
       c.Name AS CategoryName, c.Color AS CategoryColor
FROM Reports r
INNER JOIN Categories c ON c.Id = r.CategoryId
WHERE r.CreatedById = @userId
ORDER BY r.CreatedAt DESC
LIMIT @pageSize OFFSET @offset;
```

### Buscar reportes (full-text + filtros)

```
SELECT * FROM v_reports_summary
WHERE (@status IS NULL OR Status = @status)
AND (@categoryId IS NULL OR CategoryId = @categoryId)
AND (@search IS NULL OR MATCH(Title, Description, Address) AGAINST(@search IN NATURAL LANGUAGE MODE))
AND CreatedAt BETWEEN @fromDate AND @toDate
ORDER BY CreatedAt DESC;
```

## Dashboard: distribucion por estado (ultimos 30 dias)

```
SELECT Status, Last7Days, Last30Days, TotalReports, AvgAgeMinutes
FROM v_reports_by_status
ORDER BY FIELD(Status, 'Pending', 'InReview', 'Assigned', 'InProgress', 'Resolved', 'Rejected');
```

## Dashboard: carga de trabajo del staff

```
SELECT UserId, FullName, Role,
       AssignedReports, OpenReports, ResolvedReports, LastActivityAt
FROM v_staff_workload
ORDER BY OpenReports DESC, ResolvedReports DESC;
```

## Tiempo medio de resolucion por categoria

```
SELECT CategoryName, ResolvedCount, AvgMinutesToResolve, AvgHoursToResolve
FROM v_resolution_time
ORDER BY AvgMinutesToResolve ASC;
```

## Notificaciones no leidas del usuario actual

```
SELECT n.Id, n.Title, n.Body, n.Type, n.CreatedAt,
       r.Title AS ReportTitle, r.Status AS ReportStatus
FROM Notifications n
LEFT JOIN Reports r ON r.Id = n.ReportId
WHERE n.UserId = @userId AND n.IsRead = 0
ORDER BY n.CreatedAt DESC
LIMIT 20;
```

## Cambiar estado + notificar (con EF Core / Dapper)

```
// Esto NO se hace con un SP. Es un metodo de C#:
//
// public async Task<Guid> ChangeStatusAsync(
//     Guid reportId, ReportStatus newStatus, Guid actorId, string? message)
// {
//     using var tx = await _db.Database.BeginTransactionAsync();
//     var report = await _db.Reports
//         .FromSqlInterpolated($"SELECT * FROM Reports WHERE Id = {reportId} FOR UPDATE")
//         .FirstAsync();
//     var old = report.Status;
//     report.Status = newStatus;
//     if (newStatus == ReportStatus.Resolved) report.ResolvedAt = DateTime.UtcNow;
//     var update = new ReportUpdate { ... Type = StatusChange ... IsOfficial = true };
//     _db.ReportUpdates.Add(update);
//     if (report.CreatedById != actorId)
//         _db.Notifications.Add(new Notification { ... });
//     _db.AuditLogs.Add(new AuditLog { ... });
//     await _db.SaveChangesAsync();
//     await tx.CommitAsync();
//     return update.Id;
// }
```

## 6. Datos de Prueba (Seed)

El script incluye datos demo listos para presentar.

| Entidad       | Cantidad | Detalles                                                                             |
|---------------|----------|--------------------------------------------------------------------------------------|
| Categories    | 8        | Basura acumulada, Luminaria, Fuga, Bache, Infraestructura, Arbol, Senalizacion, Otro |
| Users         | 3        | ciudadano@test.com, funcionario@test.com, admin@test.com (pwd: Password123!)         |
| Reports       | 3        | Estados: InProgress, Pending, Resolved                                               |
| ReportImages  | 4        | Multiples imagenes para el reporte de basura                                         |
| ReportUpdates | 5        | StatusChange + Comment para demo                                                     |
| Notifications | 3        | StatusChanged, ReportResolved, NewComment                                            |
| DeviceTokens  | 2        | Android + iOS                                                                        |
| AuditLogs     | 3        | login_success, status_changed                                                        |

### 6.1 Credenciales de Demo

Los hashes BCrypt de la semilla son **placeholders** generados externamente. Para usarlos, el backend debe regenerar el hash al primer login, o reemplazarlos por hashes reales con un seeder que se ejecute via la app.

## 7. Estrategia de Índices

Índices diseñados para las consultas más frecuentes del backend:

| Índice                                          | Propósito                                       |
|-------------------------------------------------|-------------------------------------------------|
| idx_reports_status                              | Filtro Staff por estado                         |
| idx_reports_category                            | Filtro Staff por categoria                      |
| idx_reports_created_by                          | GET /api/reports/my                             |
| idx_reports_assigned_to                         | Carga de trabajo staff                          |
| idx_reports_created_at                          | Ordenamiento y reportes por fecha               |
| idx_reports_geo (Lat,Lng)                       | Filtros geograficos (futuro)                    |
| idx_reports_status_created (Status,CreatedAt)   | Dashboards temporales por estado                |
| ft_reports_search (FULLTEXT)                    | Busqueda full-text en Title/Description/Address |
| idx_report_updates_report (ReportId,CreatedAt)  | Timeline ordenado de un reporte                 |
| idx_notif_user_unread (UserId,IsRead,CreatedAt) | GET /api/notifications?unreadOnly=true          |
| idx_device_user_token (UserId,Token)            | Lookup rapido de token                          |
| uk_device_user_token (UserId,Token UK)          | Evitar duplicados                               |

| Índice                               | Propósito             |
|--------------------------------------|-----------------------|
| idx_audit_user (UserId, CreatedAt)   | Auditoria por usuario |
| idx_audit_action (Action, CreatedAt) | Auditoria por accion  |

## 8. Mapeo al Contrato OpenAPI

Como el esquema cumple cada endpoint del contrato streetsignal-api-contracts.yml:

| Endpoint                             | Tabla(s)                                                       |
|--------------------------------------|----------------------------------------------------------------|
| POST /api/auth/register              | Users                                                          |
| POST /api/auth/login                 | Users + RefreshTokens                                          |
| GET /api/auth/me                     | Users                                                          |
| POST /api/auth/logout                | RefreshTokens (revoke)                                         |
| GET /api/categories                  | Categories                                                     |
| POST /api/reports                    | Reports + ReportImages                                         |
| GET /api/reports/my                  | Reports (filtro CreatedById)                                   |
| GET /api/reports                     | Reports (filtros via ReportQueries LINQ)                       |
| GET /api/reports/{id}                | Reports + ReportUpdates                                        |
| PATCH /api/reports/{id}              | Reports (solo si Status=Pending)                               |
| PATCH /api/reports/{id}/status       | Reports (transaccion: update + ReportUpdate + Notification)    |
| GET /api/reports/{id}/updates        | ReportUpdates                                                  |
| POST /api/reports/{id}/updates       | ReportUpdates (transaccion: update + Notification si es staff) |
| POST /api/files/upload               | ReportImages                                                   |
| POST /api/notifications/device-token | DeviceTokens (upsert en DeviceTokenService)                    |
| GET /api/notifications               | Notifications                                                  |
| PATCH /api/notifications/{id}/read   | Notifications                                                  |

# Apéndice A. Código SQL

El script completo DDL + seed se reproduce a continuación. El archivo original es streetsignal\_db.sql. El script es idempotente: elimina y recrea la base de datos cada vez que se ejecuta.

```
-- =====
-- StreetSignal Platform - MySQL Database Schema
-- Plataforma de Reportes Ciudadanos (Mobile + .NET 8 Web API)
-- Version: 1.1.0 (simplificado para 1 día de desarrollo)
--
-- Tablas (5):
--   Users          -> Ciudadanos, funcionarios y administradores
--   Categories      -> Categorías de incidencias
--   Reports         -> Reportes ciudadanos (con imagen única + geo)
--   ReportUpdates   -> Timeline: comentarios + cambios de estado
--   Notifications   -> Bandeja de notificaciones in-app
--
-- Decisiones de simplificación (1-day MVP):
--   - Sin RefreshTokens -> JWT simple, sin rotación
--   - Sin AuditLogs     -> no necesario para demo
--   - Sin ReportImages  -> Reports.ImageUrl (1 imagen, como en OpenAPI)
--   - Sin DeviceTokens  -> sin push, polling o local notifications
--   - Sin AssignedToId  -> staff ve "todos", no hay asignación formal
--   - Sin fulltext index -> búsqueda con LIKE %texto%
--
-- Requisitos: MySQL 8.0+ o MariaDB 10.6+
-- Engine:      InnoDB (transactions, FK, full ACID)
-- Charset:     utf8mb4 / utf8mb4_unicode_ci
-- =====

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;
SET SQL_MODE           = 'STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION,NO_ZERO_DATE,NO_ZERO_IN_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_AUTO_CREATE_USER,NO_IGNORE_SPACE';
SET time_zone          = '+00:00';

-- -----
-- CREATE database if not exists (para importar sobre Railway)
-- -----
CREATE DATABASE IF NOT EXISTS railway
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE       utf8mb4_unicode_ci;

USE railway;

-- =====
-- USERS
-- =====
CREATE TABLE Users (
  Id          CHAR(36)      NOT NULL,
  FullName    VARCHAR(150) NOT NULL,
  Email       VARCHAR(150) NOT NULL,
  Phone       VARCHAR(30)   NULL,
  PasswordHash VARCHAR(255) NOT NULL COMMENT 'BCrypt(11) o Argon2id',
  Role        ENUM('Citizen','Staff','Admin') NOT NULL DEFAULT 'Citizen',
  IsActive    TINYINT(1)   NOT NULL DEFAULT 1,
  CreatedAt   DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  UpdatedAt   DATETIME(3)  NULL    ON UPDATE CURRENT_TIMESTAMP(3),

  PRIMARY KEY (Id),
  UNIQUE KEY uk_users_email (Email),
  KEY idx_users_role (Role)
) ENGINE=InnoDB
  DEFAULT CHARSET=utf8mb4
  COLLATE=utf8mb4_unicode_ci
  COMMENT='Usuarios: ciudadanos, funcionarios municipales y administradores.';

-- =====
-- CATEGORIES
-- =====
CREATE TABLE Categories (
  Id          CHAR(36)      NOT NULL,
  Name        VARCHAR(80)   NOT NULL,
  Slug        VARCHAR(80)   NOT NULL,
  Description  VARCHAR(500)  NULL,
  Icon        VARCHAR(60)   NULL COMMENT 'Nombre del icono UI (trash, lightbulb...)',
  Color       VARCHAR(20)   NULL COMMENT 'Hex #RRGGBB para badges',
  IsActive    TINYINT(1)   NOT NULL DEFAULT 1,
```

```

        SortOrder    INT                NOT NULL DEFAULT 0,
        CreatedAt    DATETIME(3)        NOT NULL DEFAULT CURRENT_TIMESTAMP(3),

        PRIMARY KEY (Id),
        UNIQUE KEY uk_categories_slug (Slug),
        KEY idx_categories_active (IsActive, SortOrder)
) ENGINE=InnoDB
  DEFAULT CHARSET=utf8mb4
  COLLATE=utf8mb4_unicode_ci
  COMMENT='Catalogo de categorias. Soft-delete via IsActive.';

-- =====
--  REPORTS  (tabla central)
-- =====
CREATE TABLE Reports (
  Id          CHAR(36)                NOT NULL,
  Title       VARCHAR(150)            NOT NULL,
  Description  TEXT                    NOT NULL,
  CategoryId  CHAR(36)                NOT NULL,
  Status      ENUM('Pending','InReview','Assigned','InProgress','Resolved','Rejected')
              NOT NULL DEFAULT 'Pending',
  Priority     ENUM('Low','Medium','High','Critical') NULL,
  Latitude    DECIMAL(10,7)           NOT NULL,
  Longitude   DECIMAL(10,7)           NOT NULL,
  Address     VARCHAR(255)            NULL,
  ImageUrl    VARCHAR(1000)           NULL COMMENT 'URL publica de la imagen (CDN, storage, etc.)',
  CreatedById CHAR(36)                NOT NULL,
  CreatedAt   DATETIME(3)             NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  UpdatedAt   DATETIME(3)             NULL ON UPDATE CURRENT_TIMESTAMP(3),
  ResolvedAt  DATETIME(3)             NULL COMMENT 'Auto-set cuando Status pasa a Resolved',

  PRIMARY KEY (Id),
  KEY idx_reports_status      (Status),
  KEY idx_reports_category   (CategoryId),
  KEY idx_reports_created_by (CreatedById),
  KEY idx_reports_created_at (CreatedAt),
  KEY idx_reports_status_date (Status, CreatedAt),
  KEY idx_reports_geo        (Latitude,

Longitude),

  CONSTRAINT fk_reports_category
    FOREIGN KEY (CategoryId) REFERENCES Categories(Id) ON DELETE RESTRICT,
  CONSTRAINT fk_reports_creator
    FOREIGN KEY (CreatedById) REFERENCES Users(Id) ON DELETE RESTRICT,

  CONSTRAINT chk_reports_lat CHECK (Latitude BETWEEN -90 AND 90),
  CONSTRAINT chk_reports_lng CHECK (Longitude BETWEEN -180 AND 180)
) ENGINE=InnoDB
  DEFAULT CHARSET=utf8mb4
  COLLATE=utf8mb4_unicode_ci
  COMMENT='Reportes ciudadanos. 1 imagen por reporte (como en el OpenAPI).';

-- =====
--  REPORT UPDATES  (timeline)
-- =====
CREATE TABLE ReportUpdates (
  Id          CHAR(36) NOT NULL,
  ReportId    CHAR(36) NOT NULL,
  CreatedById CHAR(36) NOT NULL,
  Type        ENUM('Comment','StatusChange','System') NOT NULL,
  Message     TEXT    NULL,
  OldStatus   ENUM('Pending','InReview','Assigned','InProgress','Resolved','Rejected') NULL,
  NewStatus    ENUM('Pending','InReview','Assigned','InProgress','Resolved','Rejected') NULL,
  IsOfficial  TINYINT(1) NOT NULL DEFAULT 0 COMMENT '1 si lo escribe Staff/Admin',
  CreatedAt   DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),

  PRIMARY KEY (Id),
  KEY idx_updates_report (ReportId, CreatedAt),
  KEY idx_updates_user   (CreatedById),

  CONSTRAINT fk_updates_report
    FOREIGN KEY (ReportId) REFERENCES Reports(Id) ON DELETE CASCADE,
  CONSTRAINT fk_updates_user
    FOREIGN KEY (CreatedById) REFERENCES Users(Id) ON DELETE RESTRICT,

  -- Garantiza que un StatusChange tenga OldStatus != NewStatus
  CONSTRAINT chk_updates_status_pair CHECK (
    (Type <> 'StatusChange')
    OR (OldStatus IS NOT NULL AND NewStatus IS NOT NULL AND OldStatus <> NewStatus)
  )
)

```

```

) ENGINE=InnoDB
DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci
COMMENT='Timeline inmutable: comentarios, cambios de estado, eventos sistema.';

-- =====
-- NOTIFICATIONS
-- =====
CREATE TABLE Notifications (
  Id          CHAR(36)      NOT NULL,
  UserId      CHAR(36)      NOT NULL,
  ReportId    CHAR(36)      NULL,
  Title       VARCHAR(120)  NOT NULL,
  Message     VARCHAR(500)  NOT NULL,
  IsRead      TINYINT(1)    NOT NULL DEFAULT 0,
  CreatedAt   DATETIME(3)   NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  ReadAt      DATETIME(3)   NULL,

  PRIMARY KEY (Id),
  KEY idx_notif_user_unread (UserId, IsRead, CreatedAt),

  CONSTRAINT fk_notif_user
    FOREIGN KEY (UserId) REFERENCES Users(Id) ON DELETE CASCADE,
  CONSTRAINT fk_notif_report
    FOREIGN KEY (ReportId) REFERENCES Reports(Id) ON DELETE CASCADE
) ENGINE=InnoDB
DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci
COMMENT='Bandeja de notificaciones in-app.';

-- =====
-- SEED DATA
-- Credenciales demo (password = "Password123!"):
-- ciudadano@test.com -> rol Citizen
-- funcionario@test.com -> rol Staff
-- admin@test.com -> rol Admin
-- Los hashes son placeholders BCrypt. El backend debe regenerarlos
-- al primer login, o el equipo los reemplaza con un seeder.
-- =====

-- Categorías
INSERT INTO Categories (Id, Name, Slug, Description, Icon, Color, IsActive, SortOrder) VALUES
('3f5b0a82-5f3a-4a44-8f29-8d68f6b3a001', 'Basura acumulada', 'basura-acumulada', 'Acumulacion
('3f5b0a82-5f3a-4a44-8f29-8d68f6b3a002', 'Luminaria defectuosa', 'luminaria-defectuosa', 'Farolas apa
('3f5b0a82-5f3a-4a44-8f29-8d68f6b3a003', 'Fuga de agua', 'fuga-de-agua', 'Fugas en tub
('3f5b0a82-5f3a-4a44-8f29-8d68f6b3a004', 'Dano en calle', 'dano-en-calle', 'Baches, gri
('3f5b0a82-5f3a-4a44-8f29-8d68f6b3a005', 'Infraestructura danada', 'infraestructura-danada', 'Danos en pu
('3f5b0a82-5f3a-4a44-8f29-8d68f6b3a006', 'Arbol u obstruccion', 'arbol-obstruccion', 'Arboles caid
('3f5b0a82-5f3a-4a44-8f29-8d68f6b3a007', 'Senalizacion danada', 'senalizacion-danada', 'Senales de t
('3f5b0a82-5f3a-4a44-8f29-8d68f6b3a999', 'Otro', 'otro', 'Cualquier ot

, '#64748B', 1, 99);

-- Usuarios demo
INSERT INTO Users (Id, FullName, Email, Phone, PasswordHash, Role, IsActive, CreatedAt) VALUES
('52af55ef-2a35-48cc-9e75-1ab3cf92c0c1', 'Ciudadano Demo', 'ciudadano@test.com', '8888-8888', '$2a$11
('52af55ef-2a35-48cc-9e75-1ab3cf92c0c2', 'Funcionario Demo', 'funcionario@test.com', '8888-9999', '$2a$11
('52af55ef-2a35-48cc-9e75-1ab3cf92c0c3', 'Administrador', 'admin@test.com', '8000-0000', '$2a$11

-- Reportes demo (3, segun el plan)
INSERT INTO Reports (Id, Title, Description, CategoryId, Status, Priority, Latitude, Longitude, Address,
('a6d2fd95-9d71-4b47-9627-fad5d7822001',
'Basura acumulada frente al parque central',
'Hay varias bolsas de basura acumuladas desde hace dias en la entrada norte del parque. Generan mal olo
'3f5b0a82-5f3a-4a44-8f29-8d68f6b3a001',
'InProgress', 'Medium',
10.3271000, -84.4305000,
'Ciudad Quesada, San Carlos, Alajuela',
'https://storage.example.com/reports/report-001.jpg',
'52af55ef-2a35-48cc-9e75-1ab3cf92c0c1',
'2026-06-02 10:00:00.000'),

('a6d2fd95-9d71-4b47-9627-fad5d7822002',
'Luminaria apagada en Calle 2',
'La farola del poste 145 lleva 5 noches apagada, es una zona muy transitada.',
'3f5b0a82-5f3a-4a44-8f29-8d68f6b3a002',
'Pending', 'Low',
10.3254500, -84.4321000,
'Calle 2, Ciudad Quesada',
'https://storage.example.com/reports/report-002.jpg',
'52af55ef-2a35-48cc-9e75-1ab3cf92c0c1',

```

```

'2026-06-02 11:30:00.000'),
('a6d2fd95-9d71-4b47-9627-fad5d7822003',
'Bache profundo en Avenida 6',
'Bache de aprox. 60cm de diametro y 15cm de profundidad. Ya ha danado dos vehiculos.',
'3f5b0a82-5f3a-4a44-8f29-8d68f6b3a004',
'InReview','High',
10.3300100, -84.4287000,
'Avenida 6, Ciudad Quesada',
'https://storage.example.com/reports/report-003.jpg',
'52af55ef-2a35-48cc-9e75-1ab3cf92c0c1',
'2026-05-30 14:00:00.000');

UPDATE Reports SET Status = 'Resolved', UpdatedAt = '2026-06-02 15:30:00.000', ResolvedAt = '2026-06-02
WHERE Id = 'a6d2fd95-9d71-4b47-9627-fad5d7822003';

-- Timeline de los reportes
INSERT INTO ReportUpdates (Id, ReportId, CreatedById, Type, Message, OldStatus, NewStatus, IsOfficial, C
('0cb57e24-00dd-46d6-9032-199dfd4b2001', 'a6d2fd95-9d71-4b47-9627-fad5d7822001', '52af55ef-2a35-48cc-9e7
('0cb57e24-00dd-46d6-9032-199dfd4b2002', 'a6d2fd95-9d71-4b47-9627-fad5d7822001', '52af55ef-2a35-48cc-9e7
('0cb57e24-00dd-46d6-9032-199dfd4b2003', 'a6d2fd95-9d71-4b47-9627-fad5d7822003', '52af55ef-2a35-48cc-9e7

-- Notificaciones demo
INSERT INTO Notifications (Id, UserId, ReportId, Title, Message, IsRead, CreatedAt) VALUES
('771f9b9e-cb98-4d3d-8788-955a7a442001', '52af55ef-2a35-48cc-9e75-1ab3cf92c0c1', 'a6d2fd95-9d71-4b47-962
('771f9b9e-cb98-4d3d-8788-955a7a442002', '52af55ef-2a35-48cc-9e75-1ab3cf92c0c1', 'a6d2fd95-9d71-4b47-962

-- =====
-- Verificacion final
-- =====
SELECT 'Users' AS TableName, COUNT(*) AS Rows_ FROM Users
UNION ALL SELECT 'Categories', COUNT(*) FROM Categories
UNION ALL SELECT 'Reports', COUNT(*) FROM Reports
UNION ALL SELECT 'ReportUpdates',COUNT(*) FROM ReportUpdates
UNION ALL SELECT 'Notifications',COUNT(*) FROM Notifications;

SET FOREIGN_KEY_CHECKS = 1;

```